



Tiny Second-hand Shopping Platform

📅 작성일	2026년 7월 11일
📁 과목명	시큐어코딩
🌀 과제	시작 전
≡ 이름	4기 39반 박성현

Tiny Secondhand Shopping Platform — 최종 개발 보고서

- **과목:** Secure Coding
- **프로젝트명:** tiny-secondhand-market (당근마켓/중고나라 스타일 중고거래 플랫폼)
- **기술 스택:** Flask 3.0, Flask-SQLAlchemy, Flask-Login, Flask-WTF(CSRF), Flask-Bcrypt, SQLite, Pytest, Pillow
- **GitHub Repository:** <https://github.com/tjdus2670/Tiny-Second-hand-Shopping-Platform> (Public 설정 완료)
- **AI 도구 활용:** 프로젝트 진행 전 과정(요구사항 분석, 아키텍처 설계, 보안 약점 분석 및 시큐어 코딩, pytest unit test 작성, 유지보수 트러블슈팅)에서 ChatGPT, Copilot, Claude 등 최신 AI 도구를 적극 도입·활용함.

1. 요구사항 분석

본 프로젝트는 보안약점을 최소화하는 시큐어 코딩 지침 준수와 함께, 플랫폼이 갖춰야 할 7가지 핵심 최소 요구사항을 분석하여 개발을 진행했다.

#	요구사항	구체화된 세부 요구사항	구현 상태
1	사용자 가입	회원가입/로그인, 마이페이지(소개글/비밀번호 수정), ID 중복 방지, DB 회원 관리	완료
2	상품 등록 및 조회	상품 등록(제목/가격/사진 등), 내 등록 상품 관리, 누구나 목록/상세 조회 가능	완료
3	사용자 간 소통	전체 실시간(롱폴링) 채팅, 유저 간 1:1 채팅 기능	완료
4	악성 유저/상품 차단	불량 상품/유저 신고 기능, 일정 횟수(5회) 신고 누적 시 자동 차단 및 휴면 전환	완료
5	유저 간 송금/결제	Escrow 형태의 Mock PG 가상 결제 및 구매자-판매자 간 안전 송금/거래 프로세스	완료
6	상품 검색	상품명 키워드 검색(SQLi 방지 파라미터 바인딩), 카테고리 필터링	완료
7	관리자 기능	신고 큐 처리(차단/정지/기각), 유저·상품 상태 관리, 감사 로그(Audit Log) 기록	완료

1.1 회원 관리

- 아이디(4~20자 영문/숫자/ _), 이메일 규격, 닉네임, 비밀번호(8자 이상 영문+숫자) 서버사이드 검증
- 회원가입/로그인 실패 메시지 통일을 통한 계정 열거(User Enumeration) 방지
- 비밀번호는 `bcrypt` 단방향 해시 알고리즘으로만 데이터베이스에 저장

1.2 상품 관리

- 상품 등록 시 제목, 설명, 가격, 카테고리, 상태에 대한 엄격한 서버 검증
- 이미지 파일 업로드 시 확장자/MIME 타입 위조 차단 및 디코딩 폭탄 방어 로직 적용
- 수정 및 삭제 요청 시 소유권(Owner)을 서버에서 매번 재검증하여 권한 우회 차단

1.3 사용자 간 소통 (채팅)

- 실시간에 준하는 사용자 경험 제공을 위한 **롱폴링(Long-Polling)** 기반 아키텍처 도입
- 1:1 채팅방 권한 검증 및 차단 유저 간 채팅 시작/전송 자동 차단
- DOM 기반 XSS 차단을 위해 프론트엔드 `textContent` 사용 및 strict CSP 적용

1.4 악성 유저 및 불량 상품 차단

- 신고 시 필수 사유 작성, 본인 대상 및 동일 대상 중복 신고 방지
- 신고 5회 이상 누적된 상품은 자동 차단(`blocked`), 사용자는 자동 휴면(`dormant`) 상태로 전환

1.5 유저 간 송금 및 거래 (Mock Payment & Escrow)

- 학습 및 보안 실습 환경의 특성상 실제 PG사 가맹점 계약을 대체하여 **Mock Payment Gateway**를 직접 설계

- 결제 및 송금 승인 → 거래 진행 → 구매 완료 확정 → 후기 정산에 이르는 상태 전이(State Transition) 화이트리스트 관리로 안전한 송금/거래 구조 완성

1.6 상품 검색

- SQL Injection 예방을 위한 SQLAlchemy ORM 및 파라미터 바인딩 `ilike` 처리
- 카테고리 필터링은 화이트리스트 사전 검증 처리

1.7 관리자 기능

- 관리자 계정 생성은 웹 회원가입이 아닌 CLI 커맨드(`flask make-admin`)로만 제약
- 관리자 권한 검증은 세션 캐시가 아닌 매 요청시 DB 재조회를 통한 즉시 반영(권한 회수 지연 문제 해결)

2. 시스템 설계

2.1 기능 목록

- 회원 관리**: 회원가입, 로그인, 로그아웃, 프로필 조회, 마이페이지(소개글·비밀번호 변경)
- 상품 관리**: 상품 등록, 내 상품 수정/삭제, 전체 상품 목록, 상품 상세 조회, 위시리스트(찜)
- 채팅 소통**: 전체 채팅(롱폴링), 1:1 채팅(롱폴링, 참여자 검증)
- 차단 및 제재**: 불량 상품/유저 신고, 5회 누적 시 자동 제재, 사용자 1:1 차단
- 송금 및 거래**: Mock 결제 승인, 거래 상태 변경(상태 전이 제어), 후기/매너온도 작성
- 관리자**: 신고 큐 처리(차단/정지/기각), 전체 현황 대시보드, 감사 로그(Audit Log)

2.2 웹페이지 및 라우트 설계

페이지 분류	URL 경로	설명 및 권한
회원가입	<code>/auth/register</code>	누구나 접근 가능
로그인	<code>/auth/login</code>	누구나 접근 가능 (로그인 후 세션 재발급)
마이페이지	<code>/mypage/</code>	로그인 필수
상품 목록 및 검색	<code>/products/</code>	누구나 접근 가능 (차단 상품 자동 제외)
새 상품 등록	<code>/products/new</code>	로그인 및 활성 계정 필수
상품 상세페이지	<code>/products/<id></code>	누구나 접근 가능
1:1 채팅방 목록	<code>/chat/rooms</code>	로그인 필수 (본인 참여 방안 조회)
1:1 채팅방 / 폴링	<code>/chat/rooms/<id></code> , <code>/poll</code>	참여자 검증 (IDOR 방지)
전체 채팅	<code>/chat/global</code>	로그인 필수
거래 및 결제	<code>/transactions/<id>/pay</code>	구매자 본인만 결제 접근 가능
관리자 대시보드	<code>/admin/</code>	관리자(<code>admin</code>) 권한 필수 (매 요청 DB 검증)

2.3 데이터베이스 설계

SQLite 기반으로 구축되었으며, 차단·거래·감사로그를 포함해 총 12개 테이블로 설계되어 데이터를 격리 관리한다.

users	products	transactions
id (PK)	seller_id (FK)	product_id (FK)
username (UNIQUE)	id (PK)	buyer_id (FK)
email (UNIQUE)	title, price...	seller_id (FK)
password_hash		amount, status
trust_score		mock_payment_id
status, role		
failed_login...		
	product_images	
	product_id (FK)	
	filename (UUID)	

- users**: 사용자 계정, 비밀번호 해시, 매너온도, 상태(`active` / `dormant` / `banned`), 실패 횟수, 잠금 시각
- products**: 상품 정보, 가격, 카테고리, 상태(`selling` / `reserved` / `sold` / `blocked`)
- product_images**: 상품 이미지 난수화 저장 파일명, 순서
- wishes**: 위시리스트 (user_id, product_id 복합 UNIQUE)
- chat_rooms**: 1:1 채팅방 정보 (product_id, buyer_id, seller_id)

6. `messages` : 1:1 채팅 메시지
7. `global_messages` : 전체 채팅 메시지
8. `reports` : 신고 접수 내역 (reporter_id, target_type, target_id)
9. `transactions` : 거래 및 Mock 결제 내역, 금액, idempotency_key, 상태 전이
10. `reviews` : 거래 완료 후 남긴 별점 및 후기
11. `user_blocks` : 개인 차단 내역 (blocker_id, blocked_id)
12. `admin_logs` : 관리자 조치 감사 로그 (Append-only)

2.4 시스템 아키텍처

- **Application Factory Pattern**: Flask 블루프린트(`auth`, `products`, `chat`, `reports`, `admin`, `users`, `transactions`, `mypage`) 기반 모듈화
- **롱폴링(Long-Polling) 동기화**: WebSocket 없이 표준 HTTP 기반으로 최장 25초간 서버가 대기하다 새 메시지 발생 시 즉시 응답하는 구조 구현
- **Mock Payment Gateway Engine**: 카드사 연동 모듈을 모방하여 결제 요청 → 승인 번호 발급 → 중복 결제 방지(`idempotency_key`) 트랜잭션 관리

3. 구현 상세

3.1 계정 및 인증 (Security First Authentication)

- **비밀번호 저장**: Flask-Bcrypt를 이용해 Salt가 포함된 해시값 저장
- **계정 열거(User Enumeration) 차단**: 존재하지 않는 ID이거나 비밀번호가 틀린 경우, 그리고 5회 이상 실패로 계정이 잠긴 경우 모두 동일한 오차 메시지("아이디 또는 비밀번호가 올바르지 않습니다")를 반환하여 계정 존재 여부 추측 사이드채널 완전히 방지
- **세션 안전 조치**: 로그인 성공 시 `session.clear()` 후 `login_user()` 를 호출하여 **세션 고정(Session Fixation)** 공격 차단, 쿠키에 `HttpOnly`, `SameSite=Lax` 지정

3.2 상품 관리 및 파일 업로드 보안

- **이미지 2단계 검증 및 재인코딩**:
 1. 확장자 및 MIME Header 신뢰 불가 → Pillow `Image.open()` 및 `verify()` 로 실제 이미지 구조 검증
 2. 디코딩 폭탄(Decompression Bomb) 예외를 try/except로 캡처하여 안전하게 오류 처리
 3. 이미지 리사이징(`1600x1600`) 및 RGB 재인코딩 과정에서 웹шел/폴리글랏 페이로드 소멸
 4. UUID 기반 난수 파일명으로 무작위 저장하여 Direct URL 실행 차단

3.3 사용자 간 소통 (채팅) 및 XSS 방어

- 1:1 채팅방 및 전체 채팅 메시지 수신 시 프론트엔드 DOM 컨트롤은 `textContent` 만 사용
- Jinja2 템플릿의 `safe` 필터 사용 금지 및 Strict Content Security Policy (`default-src 'self'`) 유지
- **채팅 속도 제한(Rate Limiting)**: 10초 슬라이딩 윈도우 내 5회 초과 전송 시 `HTTP 429 Too Many Requests` 에러를 반환하여 스팸 도배 방지

3.4 신고 및 자동 제재 시스템

- 상품/사용자에 대한 신고 접수 시 사유 필수 입력
- 동일 사용자의 중복 신고 및 본인 대상 신고 서버 단에서 400 에러 처리
- 신고 누적 임계치(5회) 도달 시 상품은 `blocked` 상태 전환, 사용자는 `dormant` (휴면) 계정으로 자동 전환되어 로그인 및 작성 액션 전면 차단

3.5 관리자 기능 및 감사 로그

- 대시보드 제공 및 신고 큐 처리(차단 승인/기각)
- 관리자 권한 검증은 세션 캐시가 아닌 매 요청 DB에서 `user.role == 'admin'` 및 `user.status == 'active'` 를 재조회
- 모든 조치 내역은 삭제 및 수정이 불가능한 `admin_logs` (Append-Only) 테이블에 기록

3.6 거래 및 결제 (Mock Payment & Transaction State Machine)

- 금액 조작 방지: 결제 요청 시 클라이언트가 보낸 금액을 무시하고, 서버가 DB의 `product.price` 를 직접 조회하여 금액 설정
- **상태 전이(State Transition) 화이트리스트 강제**:
`PENDING` → `PAID` → `IN_PROGRESS` → `COMPLETED` 외의 불법적인 상태 점프(예: 결제 없이 완료로 변경 등)를 모델 레벨 `ALLOWED_TRANSITIONS` 로 차단
- 역할 기반 제약: `IN_PROGRESS` 전환은 판매자만, `COMPLETED` 확정은 구매자만 가능하도록 서버 재검증

4. 보안 취약점 점검 체크리스트

#	취약점 항목	대응 여부	대응 내용 요약	검증 케이스 / 테스트
1	SQL Injection	✓	전 구간 SQLAlchemy ORM 및 파라미터 바인딩 사용	<code>test_products.py::test_search_with_sql_injection_style_input_is_safe</code>
2	XSS (Cross-Site Scripting)	✓	Jinja2 자동 이스케이프, 프론트 JS <code>textContent</code> 사용, CSP 적용	<code>test_chat.py::test_message_xss_payload_is_escaped_on_render</code>
3	CSRF	✓	Flask-WTF <code>CSRFProtect</code> 전역 적용, AJAX 헤더(<code>X-CSRFToken</code>) 토큰 검증	<code>test_csrf.py</code> (전체 4개 테스트)
4	IDOR (접근 제어)	✓	상품/이미지/채팅방/거래/마이페이지 전 구간 본인/참여자 여부 서버 재검증	<code>test_products.py</code> , <code>test_chat.py</code> , <code>test_transactions.py</code>
5	Broken Access Control	✓	관리자 권한 매 요청 DB 재조회 (권한 회수 지연 문제 방지)	<code>test_reports_admin.py</code> , <code>test_security_misc.py</code>
6	파일 업로드 취약점	✓	Pillow 실제 구조 검증 → RGB 재인코딩 → UUID 난수 파일명 저장	<code>test_products.py::test_fake_file_disguised_as_image_is_rejected</code>
7	세션 취약점	✓	HttpOnly, SameSite 쿠키, 로그인 시 세션 재발급, 5회 실패 잠금	<code>test_auth.py</code> , <code>test_security_misc.py</code>
8	개인정보 유출	✓	비밀번호 Bcrypt 해시 저장, 카드정보 미저장, 공개 프로필 화이트리스트 응답	<code>test_auth.py::test_password_is_hashed_not_stored_plaintext</code>
9	이미지 디코딩 DoS (압축 폭탄)	✓	Pillow <code>Image.open()</code> / <code>thumbnail()</code> 디코딩 구간 try-except 예외 캡처 처리	<code>test_products.py::test_decompression_bomb_image_is_rejected_gracefully</code>
10	채팅 스팸 / Rate Limit	✓	10초 내 5회 초과 메시지 전송 시 429 Too Many Requests 반환 버스트 제한 적용	<code>test_chat.py::test_first_five_rapid_room_messages_are_allowed_sixth_is_t</code>

5. 테스트 및 검증 결과

5.1 테스트 환경 및 구현

`pytest` 및 Flask Test Client를 기반으로 동작하며, 독립된 SQLite 메모리/임시 파일 DB를 생성하여 테스트 간 상호 영향이 없도록 완전 격리하여 수행했다.

5.2 pytest 실행 결과 (총 64개 테스트 케이스 100% PASS)

```
$ .\venv\Scripts\pytest.exe -v
===== test session starts =====
platform win32 -- Python 3.12.x, pytest-8.x
collected 64 items

tests/test_auth.py ..... [ 12%]
tests/test_chat.py ..... [ 26%]
```

```
tests/test_csrf.py ..... [ 32%]
tests/test_products.py ..... [ 54%]
tests/test_reports_admin.py ..... [ 67%]
tests/test_security_misc.py ..... [ 81%]
tests/test_seed.py .... [ 84%]
tests/test_transactions.py ..... [100%]

===== 64 passed in 7.42s =====
```

5.3 세부 테스트 케이스 개요

1. `test_auth.py` (8개): 회원가입, 동일 계정 중복 가입 방지, 계정 잠금, 비밀번호 변경 시 기존 비밀번호 검증, Bcrypt 해싱 확인
2. `test_chat.py` (9개): 1:1 채팅방 IDOR 접근 차단, 메시지 XSS 이스케이프, 차단 사용자 간 채팅 시작 불가, 채팅 속도 제한(Rate Limit 429) 검증
3. `test_csrf.py` (4개): CSRF 토큰 부재 시 400 반환, 유효 토큰 전송 시 성공, AJAX JSON 요청 시 `X-CSRFToken` 헤더 검증
4. `test_products.py` (14개): 로그인 필수 접근, 위장 파일 업로드 차단, SQL Injection 무력화, 상품 수정/삭제 IDOR 차단, 디코딩 폭탄 이미지 안전 거부 처리
5. `test_reports_admin.py` (8개): 신고 누락 자동 차단 및 휴면 전환, 중복 신고 방지, 비관리자 라우트 접근 403 차단, 감사 로그 기록
6. `test_security_misc.py` (9개): 위시리스트 IDOR 방지, 관리자 개별 라우트 비관리자 차단, 세션 재발급 검증, HttpOnly/SameSite 쿠키 속성 검증, 계정 존재 추측 방지 통합 메시지 검증
7. `test_seed.py` (2개): 시드 데이터 생성 및 재실행 시 중복 생성 방지 가드 검증
8. `test_transactions.py` (10개): 금액 조작 방지, 상태 전이 화이트리스트 검증, 구매자/판매자 역할별 상태 전환 권한 검증, 중복 후기 방지

6. 유지보수 및 트러블슈팅 (개발 과정 문제 해결 사례)

6.1 `sqlite3.OperationalError: unable to open database file` 문제

- 원인: `.env` 파일의 상대경로 `DATABASE_URL` 이 `config.py` 의 절대경로 기본 설정을 덮어쓰워 실행 디렉토리에 따라 DB 파일을 찾지 못함
- 조치: `.env` 에서 상대경로 `DATABASE_URL` 을 제거하고 `config.py` 에서 실행 위치 기준 절대경로로 통합 관리하도록 개선

6.2 `flask seed-demo` 데모 데이터 미생성 현상

- 원인: seed 스크립트 내 "유저가 1명이라도 존재하면 작성을 건너뛴다"는 가드가 회원가입 테스트를 먼저 수행한 경우 무조건 스킵됨
- 조치: CLI 헬퍼 명령 수정 및 안내 순서 정리

6.3 CSP 적용으로 인한 채팅 폼 제출 불가 버그

- 원인: Strict CSP(`default-src 'self'`) 환경에서 인라인인 JavaScript가 블록되어 폼 이벤트를 캡처하지 못하고 브라우저 기본 제출(새로고침)이 동작함
- 조치: `unsafe-inline` 을 허용하는 보안 완화 대신, HTML element의 `data-*` 속성에 API URL을 전달하고 외부 JS 파일에서 이를 읽어 전송하는 방식으로 시큐어 코딩 준수

6.4 롱폴링 수신 시 메시지 중복 렌더링 현상

- 원인: (a) 내가 보낸 메시지의 즉시 응답 렌더링과 롱폴링 수신 렌더링의 충돌, (b) 롱폴링 이벤트 핸들러의 중복 등록
- 조치: 프론트엔드에서 수신된 메시지 ID를 `Set` 구조로 추적하여 이중 렌더링 방지 및 `data-chat-initialized` 플래그로 중복 초기화 차단

6.5 이미지 디코딩 폭탄(Decompression Bomb) 예외 처리 보완

- 원인: 압축률이 높은 수만 픽셀 이미지 디코딩 시 Pillow에서 `DecompressionBombError` 발생하여 서버 500 내부 에러 출력 위험 존재
- 조치: `save_product_image` 함수 내에서 2단계 try/except 구문을 적용하고 `ImageValidationError` 로 예외를 래핑하여 사용자에게 정제된 에러 메시지 응답 처리 완료

7. 환경 설정 및 실행 방법 (Summary)

```
# 1. 저장소 클론 및 이동
git clone <https://github.com/사용자계명/tiny-secondhand-market.git>
cd tiny-secondhand-market

# 2. 가상환경 생성 및 활성화 (Windows PowerShell 기준)
python -m venv venv
.\venv\Scripts\activate
```

```
# 3. 의존성 패키지 설치
pip install -r requirements.txt
pip install -r requirements-dev.txt

# 4. 데모 데이터 생성 (선택 사항)
flask seed-demo

# 5. 테스트 실행 (전체 64개 통과 확인)
pytest -v

# 6. 애플리케이션 실행
flask run
```